

Lab3 AST2IRP

0. Questions

Q1.1 treep.hh中有许多 tiger_irp 的 class，他们分别起到了什么作用？

`tree::Seq`：语句序列，把多条语句按顺序串起来。

`tree::Jump`：无条件跳转到某个 label。

`tree::Cjump`：条件跳转，根据关系运算结果跳到真分支或假分支 label。

`tree::Move`：赋值语句，把 src 的值写到 dst。

`tree::Binop`：二元运算表达式，如加减乘除、逻辑/位运算。

`tree::Mem`：内存访问，表示“访问某地址处的值”。

`tree::TempExp`：把临时变量 Temp 当作表达式读取其值。

`tree::Eseq`：先执行一段语句，再计算一个表达式并把该值作为整个节点结果，常用于把副作用嵌入表达式求值过程。

`tree::Name`：把标签名当作地址常量使用。

`tree::Const`：整型常量。

`tree::Temp`：虚拟寄存器编号，承载变量在 IR 阶段的值。

`tree::Label`：控制流目标标识，用于构造非结构化跳转图。

Q1.2 相对于虎书中的Tiger IR，我们的Tiger IR+多了哪些内容，为什么需要多的这些内容？

`tree::Program`：IR 的根节点，保存整个程序的函数列表。

`tree::FuncDecl`：表示一个函数，包含函数名、参数 temps、函数体 stm、返回类型，以及本函数使用的 temp 和 label 上界信息。

`tree::Call`：方法调用表达式，显式包含对象 obj 和参数列表 args，适合面向对象语义翻译。

`tree::ExtCall`：外部函数调用表达式，用于运行库函数或系统函数。

`tree::Return`：函数返回语句，返回一个表达式值。

`tree::LabelStm`：定义一个跳转目标位置。

`tree::ExpStm`：把“有副作用但返回值不用”的表达式当语句执行。

`tree::Type`：表达式类型信息（INT 或 PTR），用于区分整数值和地址值。

引入了函数的概念和函数调用。提供更直接的函数返回与语句串关节点。

Q2

运算（算数运算、比较运算、逻辑运算.....）的翻译：见2.8/2.9

赋值：见2.6

条件（if、while）：见2.4/2.5

Q3

此次只处理了 MainMethod，直接使用的 methodname。

1. ast2tree

提供给 main 的接口。先设置 visitor 的 semant_map，再生成对应的 class_table，其中包含每个类变量和方法分配的偏移量。同名方法使用同一偏移量以实现继承覆盖和多态。

然后从 Program* 节点开始用 visitor 访问。

2. ASTToTreeVisitor::visit

2.1 Program

没有其他 ClassDecl，所以访问 main 函数即可。用一个全局变量 visit_tree_result 来传递返回值。主函数应当被翻译为 FuncDecl，将其放入函数声明列表中再打包为 Program 返回。

2.2 MainMethod

从 semant_map 获取 NameMaps，然后生成局部变量表。先处理 formal_list 中的参数，再处理 var_list 中的变量，最后处理返回参数，按顺序设置偏移量。

按顺序处理 statement 并将返回值插入 vector，最后把处理后的 vector<Stm*> 转换为 Seq*，打包为 FuncDecl 返回。

2.3 Nested

逐个处理 stm 后打包为 Seq 返回。

2.4 If

先处理条件表达式，然后将返回值生成 Tr_cx。然后分别处理 if 和 else 的 stm。新建3个标签，分别为 true/false/汇合。并将 Tr_cx 中占位的 patch_list 设置为对应标签值。最后将这些 stm 拼接起来打包为 Seq。

2.5 While

先保存外层循环的标签以确保嵌套正确。与 if 类似地，只是在最后拼接过程中的跳转不太一致。

2.6 Assign

打包为 Move。

2.7 CallStm

先递归处理参数表达式，将可求值的取出。外部调用使用 ExtCall 打包执行。返回 ExpStm。

2.8 BinaryOp

对于 && 和 ||，左右子表达式都变成 Tr_cx，生成一个中间标签 mid 来控制短路。随后将整个表达式再合成为一个大的 Tr_cx 返回。

对于其余情况，对左右子表达式求值，然后用 `unEx` 得到 `Exp`。若为关系运算符，则新建真假标签，构造 `Cjump` 并打包为 `Tr_cx` 返回。其他则为算术运算符，构造 `Binop` 并打包为 `Tr_ex` 返回。

2.9 UnaryOp

在翻译后不包含 `UnaryOp` 的类型，所以要变为其他类型。`-` 变为 `0-` 的 `BinaryOp`。`!` 用 `Tr_cx` 将 `true_list` 和 `false_list` 调换。

2.10 CallExp

和 `CallStm` 类似，但是只返回 `Tr_ex`。

2.11 Length

构造 `Binop`，用 `-` 得到一个表达式，打包为 `Tr_ex` 返回。